

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN ĐIỀU KHIỂN TỰ ĐỘNG



TIỂU LUẬN MÔN HỌC
— ĐO LƯỜNG ĐIỀU KHIỂN BẰNG MÁY TÍNH —

**XÂY DỰNG HỆ THỐNG GIÁM SÁT VÀ ĐIỀU KHIỂN THỜI
GIAN THỰC TỐC ĐỘ ĐỘNG CƠ DC QUA ETHERNET VÀ
C#/SIMULINK**

GIẢNG VIÊN HƯỚNG DẪN:

TS. NGUYỄN TRỌNG TÀI

SINH VIÊN THỰC HIỆN:

Nguyễn Văn Tuấn Anh – 2310131

Hồ Công Thành – 2313111

Cao Chí Nguyên – 2312331

TP. HỒ CHÍ MINH, 2026

Mục lục

1	TỔNG QUAN ĐỀ TÀI	1
1.1	Lý do chọn đề tài	1
1.2	Mục tiêu đề tài	2
1.3	Đối tượng và phạm vi nghiên cứu	2
1.4	Sơ đồ tổng quan hệ thống	3
2	CƠ SỞ LÝ THUYẾT VỀ ETHERNET VÀ TCP/IP	4
2.1	Tổng quan về Ethernet	4
2.2	Mô hình TCP/IP	5
2.3	Cấu trúc Frame truyền của hệ thống	5
3	MẠCH NGUYÊN LÝ VÀ LAYOUT	7
3.1	Sơ đồ nguyên lí	7
3.2	PCB Layout	8
4	THIẾT KẾ FIRMWARE HỆ THỐNG	10
4.1	Tổng quan cấu trúc firmware	10
4.2	Khởi tạo các module phần cứng	11
4.2.1	Khởi tạo Encoder	11
4.2.2	Khởi tạo điều khiển động cơ	11
4.2.3	Khởi tạo Ethernet W5500	13
4.3	Các hàm xử lý trong firmware	14

4.3.1	Hàm điều khiển tốc độ động cơ	14
4.3.2	Hàm điều khiển chiều quay động cơ	14
4.3.3	Hàm tính tốc độ RPM	15
4.3.4	Task điều khiển PI	15
4.3.5	Hàm kiểm tra CRC	17
4.3.6	Hàm xử lý TCP Server	17
4.3.7	Thiết kế frame truyền thông trong firmware	18
4.4	Luồng hoạt động của firmware	19
5	MATLAB SIMULINK – PHÂN TÍCH MÃ NGUỒN CƠ CHẾ GIAO TIẾP VÀ THU THẬP DỮ LIỆU QUA TCP/IP	21
5.1	Khởi tạo cấu hình và thiết lập kết nối TCP/IP	21
5.2	Vòng lặp lấy mẫu thời gian	22
6	THIẾT KẾ GIAO DIỆN QUẢN LÝ C#	24
6.1	Thiết kế giao diện winform	25
6.2	Cấu hình TCP Client trên C#	25
6.3	Xây dựng hàm đóng gói frame @SEN	26
6.4	Chương trình con tính CRC	27
6.5	Chương trình con đóng gói frame @SEN	27
6.6	Chương trình con gửi frame xuống ESP32	28
6.7	Xây dựng hàm nhận và giải mã frame @REC	29
6.8	Chương trình con nhận dữ liệu từ ESP32	29
6.9	Chương trình con xử lý bộ đệm nhận	30
6.10	Chương trình con kiểm tra frame nhận	31
6.11	Chương trình con giải mã frame nhận	31
6.12	Hiển thị dữ liệu realtime: tốc độ, PWM, trạng thái kết nối	32
6.13	Chương trình con cập nhật trạng thái kết nối	32
6.14	Chương trình con ghi log realtime	33

6.15	Chương trình con cập nhật tốc độ RPM	33
6.16	Chương trình con cập nhật giá trị PWM	33
6.17	Chương trình con đọc tốc độ RPM	34
6.18	Chương trình con tự động đọc RPM	34
6.19	Đoạn xử lý dữ liệu realtime trong frame phản hồi @REC	34
7	TỔNG KẾT	36
8	TÀI LIỆU THAM KHẢO	38

Danh sách hình vẽ

2.1	Figure 21: Cấu trúc tổng quát	5
2.2	Figure 22: Cấu trúc frame truyền của Ethernet	5
2.3	Figure 23: Cấu trúc frame truyền của IP Package	6
2.4	Figure 24: Cấu trúc frame truyền của TCP segment	6
3.1	Hình 4.1: Sơ đồ kết nối chân của ESP32	7
3.2	Hình 4.2: Sơ đồ kết nối chân của module W5500	8
3.3	Hình 4.3: Sơ đồ kết nối chân để kết nối với L298N	8
3.4	Hình 4.4: Sơ đồ kết nối chân của L298N	8
3.5	Hình 4.5: Sơ đồ kết nối chân để kết nối với DC Servo	8
3.6	Hình 4.6: Sơ đồ kết nối chân của DC Servo	8
3.7	Hình 4.7: PCB Layout	8
3.8	Hình 4.8: 3D View	9
4.1	Hình 3.1. Môi trường phát triển firmware ESP32 sử dụng ESP-IDF trên Visual Studio Code	10
4.2	Hình 3.2. Sơ đồ cấu trúc tổng quan firmware ESP32.	10
5.1	Hình 5: Đồ thị đáp ứng tốc độ của động cơ	23
6.1	Figure 61: Tổng quan giao diện C#	25
6.2	Figure 62: Kết quả đọc log	33
6.3	Figure 63: Cập nhật PWM manual	34
6.4	Figure 64: Dữ liệu rpm realtime	34

Danh sách bảng

Chương 1

TỔNG QUAN ĐỀ TÀI

1.1 Lý do chọn đề tài

Trong các hệ thống điều khiển và giám sát hiện đại, việc truyền dữ liệu giữa thiết bị điều khiển và máy tính giám sát đóng vai trò rất quan trọng. Các chuẩn giao tiếp truyền thống như UART, USB hoặc truyền thông không dây có thể đáp ứng tốt trong các ứng dụng đơn giản, tuy nhiên khi hệ thống yêu cầu khoảng cách truyền xa hơn, tốc độ ổn định hơn và khả năng kết nối trong mạng LAN thì Ethernet là một lựa chọn phù hợp.

Ethernet là chuẩn truyền thông mạng có dây được sử dụng phổ biến trong công nghiệp, phòng thí nghiệm, hệ thống đo lường và điều khiển. Ưu điểm của Ethernet là khả năng truyền dữ liệu ổn định, tốc độ cao, dễ mở rộng và có thể kết nối nhiều thiết bị trong cùng một mạng. Trên nền Ethernet, các giao thức TCP/IP cho phép các thiết bị giao tiếp với nhau thông qua địa chỉ IP và port, giúp việc xây dựng hệ thống điều khiển từ máy tính trở nên thuận tiện hơn.

ESP32 là một vi điều khiển có hiệu năng tốt, tích hợp nhiều ngoại vi như GPIO, ADC, PWM, SPI, UART, WiFi và Bluetooth. Tuy nhiên, ESP32 không có sẵn cổng Ethernet RJ45 trên các board phát triển thông dụng. Vì vậy, việc kết hợp ESP32 với module Ethernet W5500 thông qua giao tiếp SPI cho phép xây dựng một card giao tiếp Ethernet có khả năng nhận lệnh từ máy tính, xử lý dữ liệu và điều khiển thiết bị ngoại vi.

Trong đề tài này, nhóm thực hiện xây dựng một hệ thống card giao tiếp Ethernet sử dụng ESP32 và W5500. Hệ thống cho phép phần mềm trên máy tính giao tiếp với ESP32 thông qua mạng TCP/IP. Dữ liệu truyền nhận giữa máy tính và ESP32 được đóng gói theo frame cố định 12 byte với định dạng @SEN cho frame truyền đi và @REC cho frame phản hồi. Ngoài ra, hệ thống được ứng dụng để điều khiển động cơ DC có encoder, đọc

tốc độ phản hồi và triển khai bộ điều khiển PID thông qua C#, Matlab/Simulink hoặc các công cụ giao tiếp thiết bị như VISA.

Vì vậy, đề tài có ý nghĩa trong việc kết hợp kiến thức về vi điều khiển, truyền thông Ethernet, TCP/IP, lập trình giao diện C#, thiết kế giao thức truyền thông và điều khiển hệ thống thực tế.

1.2 Mục tiêu đề tài

Mục tiêu chính của đề tài là thiết kế và xây dựng một hệ thống giao tiếp Ethernet sử dụng ESP32 và module W5500, cho phép máy tính truyền nhận dữ liệu với ESP32 thông qua giao thức TCP/IP.

Các mục tiêu cụ thể gồm:

Tìm hiểu mô hình TCP/IP và vai trò của Ethernet trong hệ thống truyền thông mạng.

Thiết kế card giao tiếp Ethernet sử dụng ESP32 kết hợp module W5500.

Cấu hình ESP32 giao tiếp với W5500 thông qua chuẩn SPI.

Xây dựng ESP32 đóng vai trò TCP Server để nhận kết nối từ máy tính.

Xây dựng phần mềm C# WinForms đóng vai trò TCP Client để kết nối, gửi lệnh và nhận dữ liệu từ ESP32.

Thiết kế frame truyền thông cố định 12 byte với định dạng @SEN và @REC.

Xây dựng cơ chế kiểm tra dữ liệu bằng mã kiểm tra CRC/XOR.

Điều khiển các tín hiệu ngoại vi như LED, PWM, ADC và encoder.

Ứng dụng hệ thống vào điều khiển tốc độ động cơ DC có encoder.

Khảo sát khả năng kết nối với Matlab/Simulink hoặc VISA để phục vụ mô phỏng, giám sát và điều khiển.

1.3 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài là hệ thống truyền thông Ethernet giữa máy tính và vi điều khiển ESP32 thông qua module W5500. Trong đó, ESP32 đóng vai trò thiết bị điều khiển và thu thập dữ liệu, còn máy tính đóng vai trò giao diện giám sát, điều khiển và xử lý dữ liệu.

Phạm vi nghiên cứu của đề tài bao gồm:

Truyền thông Ethernet trong mạng LAN.

Giao thức TCP/IP giữa máy tính và ESP32.

Giao tiếp SPI giữa ESP32 và W5500.

Lập trình TCP Server trên ESP32.

Lập trình TCP Client trên C# WinForms.

Thiết kế frame truyền thông cố định 12 byte.

Điều khiển PWM, đọc ADC, đọc encoder và điều khiển động cơ DC.

Ứng dụng bộ điều khiển PID trong điều khiển tốc độ động cơ.

Đề tài tập trung vào việc xây dựng hệ thống giao tiếp và điều khiển ở mức mô hình thực nghiệm. Các vấn đề chuyên sâu về thiết kế mạch in công nghiệp, bảo mật mạng, tối ưu thời gian thực ở mức hệ điều hành thời gian thực hoặc triển khai hệ thống công nghiệp quy mô lớn không thuộc phạm vi chính của đề tài.

1.4 Sơ đồ tổng quan hệ thống

Chương 2

CƠ SỞ LÝ THUYẾT VỀ ETHERNET VÀ TCP/IP

2.1 Tổng quan về Ethernet

Ethernet là công nghệ truyền thông mạng cục bộ có dây, thường được sử dụng trong các hệ thống mạng LAN. Ethernet quy định cách các thiết bị truyền dữ liệu qua môi trường vật lý như cáp xoắn đôi, đầu nối RJ45, switch mạng và card mạng.

Trong đề tài, ESP32 không có sẵn cổng Ethernet RJ45 trên board phát triển thông dụng, vì vậy cần sử dụng thêm module Ethernet W5500. Module W5500 đóng vai trò là bộ điều khiển Ethernet, giúp ESP32 có thể kết nối vào mạng LAN thông qua cổng RJ45. ESP32 giao tiếp với W5500 bằng chuẩn SPI.

Ethernet đảm nhiệm các chức năng chính sau:

Cung cấp môi trường truyền dữ liệu vật lý qua dây mạng RJ45.

Truyền dữ liệu trong phạm vi mạng LAN.

Sử dụng địa chỉ MAC để nhận diện thiết bị trong mạng cục bộ.

Đóng gói dữ liệu ở tầng thấp dưới dạng Ethernet frame.

Là nền tảng để các giao thức tầng trên như IP, TCP và UDP hoạt động.

Trong hệ thống thực tế, phía ESP32 sử dụng Ethernet thông qua W5500. Phía máy tính có thể kết nối vào router bằng Ethernet hoặc WiFi. Nếu máy tính dùng WiFi còn ESP32 dùng dây Ethernet, hai thiết bị vẫn có thể giao tiếp được nếu chúng nằm trong cùng một mạng LAN.

2.2 Mô hình TCP/IP

TCP/IP là bộ giao thức nền tảng được sử dụng trong truyền thông mạng. Mô hình TCP/IP thường được mô tả gồm 4 tầng chính:

Tầng ứng dụng - Application Layer

C#

Tầng giao vận - Transport Layer

TCP/UDP

Tầng Internet - Internet Layer

IP

Tầng liên kết mạng- Network Access / Link Layer

Ethernet

Trong một số tài liệu, tầng thấp nhất có thể được gọi là tầng vật lý hoặc tầng liên kết mạng. Về bản chất, tầng này bao gồm các công nghệ truyền dữ liệu ở mức thấp như Ethernet hoặc WiFi.

Mô hình TCP/IP trong hệ thống của đề tài có thể biểu diễn như sau:

Khi C# gửi một frame điều khiển xuống ESP32, frame này không đi trực tiếp trên dây Ethernet. Thay vào đó, dữ liệu được đóng gói qua nhiều lớp:

[Ethernet Frame [IP Packet [TCP Segment [@SEN ... #]]]]

Trong đó, @SEN ... # là dữ liệu ứng dụng do nhóm thiết kế, TCP đảm bảo dữ liệu được truyền ổn định, IP định tuyến dữ liệu tới đúng địa chỉ thiết bị, còn Ethernet truyền dữ liệu qua mạng vật lý.

2.3 Cấu trúc Frame truyền của hệ thống

Hình 2.1: Figure 21: Cấu trúc tổng quát

Hình 2.2: Figure 22: Cấu trúc frame truyền của Ethernet

Hình 2.3: Figure 23: Cấu trúc frame truyền của IP Package

Hình 2.4: Figure 24: Cấu trúc frame truyền của TCP segment

Chương 3

MẠCH NGUYÊN LÝ VÀ LAYOUT

3.1 Sơ đồ nguyên lí

Hình 3.1: Hình 4.1: Sơ đồ kết nối chân của ESP32

Cụm giao tiếp SPI với khối Ethernet W5500 (MO, MI, SCK, CS)

Các chân được chọn: IO23 (MOSI), IO19 (MISO), IO18 (SCK), IO5 (CS).

Lý do: Đây là cụm chân mặc định của bộ VSPI (Hardware SPI) trên ESP32.

ESP32 có thể map SPI ra bất kỳ chân nào thông qua GPIO Matrix (phần mềm), nhưng việc sử dụng cụm chân Hardware SPI mặc định giúp đường truyền đạt tốc độ tối đa (có thể lên tới 80MHz) mà không bị độ trễ (latency).

Thư viện `esp_eth` mặc định cũng sẽ tìm kiếm các chân này đầu tiên, giúp code của bạn gọn gàng và ít lỗi khởi tạo nhất.

Cụm điều khiển động cơ L298N (IN1, IN2, ENA)

Các chân được chọn: IO26 (IN1), IO25 (IN2), IO27 (ENA).

Lý do: Bộ điều khiển băm xung (LEDC) của ESP32 có thể xuất PWM ở nhiều chân, nhưng IO25, 26, 27 là các chân GPIO "sạch". Nghĩa là chúng không tham gia vào quá trình khởi động (Boot) của chip.

Khi bạn vừa cấp nguồn cho ESP32, các chân này sẽ nằm im ở mức LOW. Điều này cực kỳ an toàn cho mạch công suất L298N, giúp động cơ không bị giật hay tự động rô lên trong lúc chờ chip nạp code xong.

Cụm đọc tín hiệu đếm Encoder (Phase A, Phase B)

Các chân được chọn: IO32, IO33.

Lý do: Khối Pulse Counter (PCNT) cần những chân đọc tín hiệu đầu vào có độ ổn định cao. IO32 và IO33 thuộc tập hợp chân ADC1, được thiết kế với trở kháng đầu vào rất tốt.

So với các chân GPIO từ 34 đến 39 (vốn chỉ có thể làm Input và thiếu điện trở kéo nội bộ), thì IO32 và IO33 là các chân đa năng, có hỗ trợ Pull-up/Pull-down nội bộ (Internal Resistors), giúp tín hiệu xung vuông từ Encoder đưa về được "vuông vức" và ít nhiễu hơn.

Hình 3.2: Hình 4.2: Sơ đồ kết nối chân của module W5500

Hình 3.3: Hình 4.3: Sơ đồ kết nối chân để kết nối với L298N

Hình 3.4: Hình 4.4: Sơ đồ kết nối chân của L298N

Hình 3.5: Hình 4.5: Sơ đồ kết nối chân để kết nối với DC Servo

Hình 3.6: Hình 4.6: Sơ đồ kết nối chân của DC Servo

Vì lý do không thể kiểm được thư viện của module L298N và động cơ DC Servo có tích hợp Encoder, nên khi vẽ mạch PCB thì ta chỉ có thể kéo ra các chân của header.

3.2 PCB Layout

Hình 3.7: Hình 4.7: PCB Layout

Hình 3.8: Hình 4.8: 3D View

Chương 4

THIẾT KẾ FIRMWARE HỆ THỐNG

4.1 Tổng quan cấu trúc firmware

Firmware của hệ thống được xây dựng trên nền tảng ESP-IDF và chạy trên vi điều khiển ESP32. Chương trình có nhiệm vụ điều khiển động cơ DC thông qua module L298N, đọc tín hiệu phản hồi tốc độ từ encoder, xử lý thuật toán điều khiển PI và giao tiếp với phần mềm máy tính thông qua chuẩn Ethernet TCP/IP sử dụng module W5500.

Hình 4.1: Hình 3.1. Môi trường phát triển firmware ESP32 sử dụng ESP-IDF trên Visual Studio Code

Firmware được tổ chức theo mô hình đa nhiệm của FreeRTOS. Trong đó, các chức năng chính như đo tốc độ, điều khiển động cơ và truyền thông TCP/IP được tách thành các khối xử lý riêng. Việc chia chương trình thành nhiều khối giúp hệ thống hoạt động ổn định, dễ mở rộng và thuận tiện cho quá trình kiểm tra, sửa lỗi.

Cấu trúc firmware gồm các khối chức năng chính: khối khởi tạo phần cứng, khối đọc encoder, khối điều khiển PWM cho động cơ, khối điều khiển PI, khối truyền thông Ethernet TCP/IP, khối xử lý frame dữ liệu và khối phản hồi dữ liệu về máy tính.

Hình 4.2: Hình 3.2. Sơ đồ cấu trúc tổng quan firmware ESP32.

4.2 Khởi tạo các module phần cứng

4.2.1 Khởi tạo Encoder

Encoder được sử dụng để đo tốc độ quay thực tế của động cơ DC. Trong chương trình, ESP32 sử dụng bộ đếm xung PCNT để đọc tín hiệu từ encoder. Tín hiệu encoder được đưa vào chân GPIO32. Mỗi khi có cạnh xung xuất hiện, bộ PCNT sẽ tăng giá trị bộ đếm.

Chương trình khởi tạo encoder:

```
void init_encoder(void) {
    ESP_LOGI(TAG, "Khởi tạo PCNT chỉ đọc kênh A (GPIO 32)");
    pcnt_unit_config_t unit_config = {
        .high_limit = 32767,
        .low_limit = -32768,
    };
    ESP_ERROR_CHECK(pcnt_new_unit(&unit_config, &pcnt_unit));
    pcnt_chan_config_t chan_config = {
        .edge_gpio_num = 32, // Sử dụng chân A
        .level_gpio_num = -1, // Không kết nối chân B
    };
    pcnt_channel_config(&pcnt_unit, 0, &chan_config);
}
```

Trong đoạn code trên, chân GPIO32 được cấu hình làm ngõ vào đọc xung encoder.

Tham số `level_gpio_num = -1` cho biết kênh không sử dụng chân B của encoder. Vì vậy, chương trình chỉ đo được số xung và tốc độ quay, chưa xác định được chiều quay của động cơ.

Thông số số xung trên mỗi vòng quay được khai báo như sau:

```
#define PULSES_PER_REV 250
```

Giá trị này được dùng trong công thức tính tốc độ RPM. Nếu encoder thực tế có số xung khác, cần thay đổi lại thông số này để kết quả đo chính xác.

4.2.2 Khởi tạo điều khiển động cơ

Động cơ DC được điều khiển thông qua module công suất L298N. ESP32 sử dụng hai chân GPIO25 và GPIO26 để điều khiển chiều quay, đồng thời sử dụng chân GPIO27 để xuất tín hiệu PWM điều chỉnh tốc độ động cơ.

Các chân điều khiển động cơ được khai báo như sau:

```
#define MOTOR_IN1_PIN 26
#define MOTOR_IN2_PIN 25
#define MOTOR_ENA_PIN 27
```

Trong chương trình, PWM được tạo bằng bộ LEDC của ESP32. Tần số PWM được đặt là 5 kHz, độ phân giải 8 bit, tương ứng với dải giá trị điều khiển từ 0 đến 255. Khi giá trị PWM bằng 0, động cơ dừng. Khi giá trị PWM tăng, điện áp trung bình cấp cho động cơ tăng, làm tốc độ động cơ tăng theo.

Cấu hình các hằng số PWM cho động cơ:

```
#define MOTOR_PWM_DUTY_RES LEDC_TIMER_8_BIT // 8 bit: 0 -> 255
#define MOTOR_PWM_FREQ_HZ 5000
#define MOTOR_PWM_MAX_DUTY 255
```

Chương trình khởi tạo PWM:

```
void init_motor(void) {
    ESP_LOGI(TAG, "Khởi tạo Motor Driver L298N (IN1=26, IN2=25, ENA=27)");
    // Cấu hình chân điều khiển quay và Output
    gpio_config_t motor_gpio_conf = {
        .pin_bit_mask = (1ULL << MOTOR_IN1_PIN) | (1ULL << MOTOR_IN2_PIN),
        .mode = GPIO_MODE_OUTPUT,
        .pull_up_en = GPIO_PULLUP_DISABLE,
        .pull_down_en = GPIO_PULLEDOWN_DISABLE,
        .intr_type = GPIO_INTR_DISABLE
    };
    ESP_ERROR_CHECK(gpio_config(&motor_gpio_conf));
    // Cấu hình Timer PWM: 5 kHz, 8 bit, duty 0 -> 255
    ledc_timer_config_t ledc_timer = {
        .speed_mode = MOTOR_PWM_MODE,
        .timer_num = MOTOR_PWM_TIMER,
        .duty_resolution = MOTOR_PWM_DUTY_RES,
        .freq_hz = MOTOR_PWM_FREQ_HZ,
        .clk_cfg = LEDC_AUTO_CLK
    };
    ESP_ERROR_CHECK(ledc_timer_config(&ledc_timer));
    // Cấu hình chân PWM xuất ra chân ENA = GPIO27
```

```

ledc_channel_config_t ledc_channel = {
    .speed_mode = MOTOR_PWM_MODE,
    .channel     = MOTOR_PWM_CHANNEL,
    .timer_sel   = MOTOR_PWM_TIMER,
    .intr_type   = LEDC_INTR_DISABLE,
    .gpio_num    = MOTOR_ENA_PIN,
    .duty        = 0,
    .hpoint      = 0
};
ESP_ERROR_CHECK(ledc_channel_config(&ledc_channel));

// ặMc định ừđng đợc khi ởkhi đợc
gpio_set_level(MOTOR_IN1_PIN, 0);
gpio_set_level(MOTOR_IN2_PIN, 0);
set_motor_speed(0);
}

```

Chiều quay thuận của động cơ được thiết lập bằng cách cho $IN1 = 1$ và $IN2 = 0$. Khi cần dừng động cơ, chương trình đưa cả hai chân $IN1$ và $IN2$ về mức 0, đồng thời đặt PWM bằng 0.

4.2.3 Khởi tạo Ethernet W5500

Module Ethernet W5500 được sử dụng để kết nối ESP32 với máy tính thông qua mạng TCP/IP. Trong chương trình, W5500 được khởi tạo bằng hàm cấu hình Ethernet có sẵn trong ESP-IDF. Sau khi khởi tạo thành công, ESP32 sẽ bắt đầu kết nối mạng và nhận địa chỉ IP.

Khi Ethernet hoạt động, chương trình sẽ ghi nhận các sự kiện như Ethernet Started, Ethernet Link Up, Ethernet Link Down và Ethernet Got IP. Địa chỉ IP nhận được sẽ được in ra màn hình Serial Monitor để người dùng biết địa chỉ cần kết nối từ phần mềm máy tính.

4.3 Các hàm xử lý trong firmware

4.3.1 Hàm điều khiển tốc độ động cơ

Hàm điều khiển tốc độ động cơ có nhiệm vụ giới hạn và xuất giá trị PWM ra chân điều khiển ENA của L298N. Giá trị PWM đầu vào được giới hạn trong khoảng từ 0 đến 255 để phù hợp với độ phân giải 8 bit của bộ PWM.

Chương trình điều khiển tốc độ động cơ:

```
static void set_motor_speed(int speed)
{
    if (speed < 0) {
        speed = 0;
    }
    if (speed > MOTOR_PWM_MAX_DUTY) {
        speed = MOTOR_PWM_MAX_DUTY;
    }
    ledc_set_duty(MOTOR_PWM_MODE, MOTOR_PWM_CHANNEL, speed);
    ledc_update_duty(MOTOR_PWM_MODE, MOTOR_PWM_CHANNEL);
    g_current_pwm = speed;
}
```

Sau khi giới hạn giá trị PWM, chương trình cập nhật duty cycle cho bộ LEDC. Giá trị PWM hiện tại cũng được lưu vào biến `g_current_pwm` để phục vụ việc giám sát và phản hồi về máy tính.

4.3.2 Hàm điều khiển chiều quay động cơ

Hàm điều khiển chiều quay động cơ trong chương trình hiện tại được thiết kế cho chiều quay thuận. Khi giá trị PWM lớn hơn 0, chương trình đặt $IN1 = 1$ và $IN2 = 0$, sau đó xuất PWM ra chân ENA. Khi PWM bằng 0 hoặc nhỏ hơn 0, chương trình dừng động cơ bằng cách đưa $IN1 = 0$, $IN2 = 0$ và $PWM = 0$.

Việc tách riêng hàm điều khiển chiều quay giúp chương trình dễ mở rộng. Nếu cần điều khiển động cơ quay nghịch, có thể bổ sung thêm chế độ $IN1 = 0$ và $IN2 = 1$.

Hàm điều khiển chiều quay động cơ:

```
static void motor_drive_forward(int pwm)
```

```

{
    if (pwm <= 0) {
        gpio_set_level(MOTOR_IN1_PIN, 0);
        gpio_set_level(MOTOR_IN2_PIN, 0);
        set_motor_speed(0);
        return;
    }
    gpio_set_level(MOTOR_IN1_PIN, 1);
    gpio_set_level(MOTOR_IN2_PIN, 0);
}

```

4.3.3 Hàm tính tốc độ RPM

Tốc độ động cơ được tính dựa trên số xung encoder thu được trong một khoảng thời gian. Chương trình đọc giá trị bộ đếm hiện tại của PCNT, sau đó so sánh với giá trị bộ đếm ở lần đọc trước để tính số xung tăng thêm.

Công thức tính tốc độ quay như sau:

$$\text{RPM} = (\text{số xung đo được} / \text{số xung trên một vòng}) / \text{Thời gian tính theo phút}$$

Trong đó, số xung trên một vòng được khai báo là 250 xung/vòng. Thời gian được đo bằng hàm thời gian của ESP32 với đơn vị micro giây, sau đó đổi sang phút. Kết quả cuối cùng là tốc độ động cơ tính theo vòng/phút.

Hàm tính RPM từ xung đọc được từ encoder:

```

float delta_pulse = (float)(current_count - last_count);
float delta_time_min = (float)delta_time_us / 60000000.0f;
int rpm = (int)((delta_pulse / (float)PULSES_PER_REV) / delta_time_min);
last_count = current_count;
last_time = current_time;

```

4.3.4 Task điều khiển PI

Task điều khiển PI được chạy định kỳ sau mỗi 100 ms. Nhiệm vụ đầu tiên của task là cập nhật giá trị tốc độ thực tế của động cơ thông qua hàm tính RPM. Sau đó, nếu chế độ PI được bật, chương trình sẽ so sánh tốc độ đặt với tốc độ thực tế để tính sai lệch.

Sai lệch tốc độ được tính theo công thức:

Bộ điều khiển PI sử dụng hai thành phần chính là thành phần tỉ lệ Kp và thành phần tích phân Ki. Thành phần tỉ lệ giúp hệ thống phản ứng nhanh với sai lệch tốc độ, còn thành phần tích phân giúp giảm sai số xác lập.

Giá trị điều khiển được tính theo dạng:

Chương trình PI:

```
float error = (float)(g_target_rpm - g_rpm);
float next_integral = g_pi_integral + error * dt;
float output = g_kp * error + g_ki * next_integral;
```

Sau khi tính toán, giá trị PWM được giới hạn trong khoảng 0 đến 255 trước khi xuất ra động cơ. Nếu PWM quá nhỏ nhưng vẫn lớn hơn 0, chương trình ép PWM lên một giá trị tối thiểu để tránh trường hợp động cơ không thắng được ma sát ban đầu.

Chương trình giới hạn tín hiệu điều khiển PWM:

```
if (output > MOTOR_PWM_MAX_DUTY) {
    output = MOTOR_PWM_MAX_DUTY;
} else if (output < 0.0f) {
    output = 0.0f;
} else {
    g_pi_integral = next_integral;
}
int pwm = (int)(output + 0.5f);
if (pwm > 0 && pwm < MOTOR_PWM_MIN_RUN) {
    pwm = MOTOR_PWM_MIN_RUN;
}
motor_drive_forward(pwm);
}
```

Ngoài ra, chương trình có thêm cơ chế khởi động nhanh. Khi động cơ đang đứng yên và có yêu cầu chạy, hệ thống sẽ cấp một giá trị PWM lớn trong một khoảng thời gian ngắn để giúp động cơ khởi động. Sau đó, bộ điều khiển PI mới bắt đầu điều chỉnh PWM theo tốc độ thực tế.

4.3.5 Hàm kiểm tra CRC

Để hạn chế lỗi trong quá trình truyền nhận dữ liệu, firmware sử dụng một byte kiểm tra lỗi dạng XOR checksum. Byte kiểm tra này được tính bằng cách XOR các byte từ vị trí 0 đến vị trí 9 trong frame dữ liệu.

Chương trình kiểm tra CRC:

```
uint8_t calc_crc(uint8_t *frame) {  
    uint8_t crc = 0;  
    for (int i = 0; i <= 9; i++) {  
        crc ^= frame[i];  
    }  
    return crc;  
}
```

Công thức kiểm tra có thể biểu diễn như sau:

Khi ESP32 nhận được frame từ máy tính, chương trình sẽ tính lại giá trị CRC và so sánh với byte CRC được gửi kèm trong frame. Nếu hai giá trị không giống nhau, frame được xem là lỗi và chương trình sẽ bỏ qua, không thực hiện lệnh điều khiển. Cơ chế này giúp tránh trường hợp động cơ hoạt động sai do dữ liệu truyền bị nhiễu hoặc sai định dạng.

4.3.6 Hàm xử lý TCP Server

ESP32 được cấu hình đóng vai trò TCP Server thông qua module Ethernet W5500. Server mở socket và lắng nghe kết nối tại cổng 5000. Phần mềm C# trên máy tính đóng vai trò TCP Client và kết nối đến địa chỉ IP của ESP32. Sau khi client kết nối thành công, ESP32 liên tục chờ nhận dữ liệu bằng hàm `recv()`. Dữ liệu nhận được được lưu vào bộ đệm `rx_buffer` để chuyển sang bước kiểm tra và xử lý lệnh. Khi dữ liệu hợp lệ, chương trình xác định mã lệnh CMD và thực hiện chức năng tương ứng như ghi PWM, đặt tốc độ RPM, cập nhật hệ số PI hoặc đọc dữ liệu phản hồi. Sau khi xử lý xong, ESP32 tạo dữ liệu phản hồi và gửi lại cho máy tính bằng hàm `send()`. Nếu client ngắt kết nối, chương trình tự động dừng động cơ bằng cách đưa PWM về 0 và đặt các chân điều khiển về mức thấp. Điều này giúp đảm bảo an toàn cho hệ thống khi mất kết nối truyền thông.

4.3.7 Thiết kế frame truyền thông trong firmware

Dữ liệu giữa máy tính và ESP32 được đóng gói theo frame cố định 12 byte. Việc sử dụng frame cố định giúp chương trình dễ kiểm tra, dễ giải mã và giảm lỗi khi truyền nhận dữ liệu.

Frame máy tính gửi xuống ESP32 có dạng:

0

123

4

5

6

7

8

9

10

11

@

S E N

CMD

DATA_H

DATA_L

OPT1

OPT2

SEQ

CRC

#

Trong đó, “@” là ký tự bắt đầu frame, “SEN” cho biết đây là frame gửi từ máy tính xuống ESP32, CMD là mã lệnh, DATA_H và DATA_L là hai byte dữ liệu, OPT1 và OPT2 là hai byte dự phòng, SEQ là số thứ tự gói tin, CRC là byte kiểm tra lỗi và “#” là

ký tự kết thúc frame.

Frame phản hồi từ ESP32 gửi về máy tính có dạng:

@

R E C

CMD

DATA_H

DATA_L

STATUS

ERROR

SEQ

CRC

#

Trong đó, “REC” cho biết đây là frame phản hồi từ ESP32. CMD là mã lệnh được phản hồi, DATA_H và DATA_L chứa dữ liệu trả về, STATUS cho biết trạng thái xử lý, ERROR cho biết mã lỗi nếu có, SEQ là số thứ tự gói tin và CRC là byte kiểm tra lỗi.

Các mã lệnh chính trong firmware gồm: lệnh ghi PWM, lệnh đọc tốc độ encoder, lệnh đặt tốc độ mong muốn, lệnh đặt hệ số Kp, lệnh đặt hệ số Ki, lệnh bật/tắt bộ điều khiển PI và lệnh đọc giá trị PWM hiện tại.

4.4 Luồng hoạt động của firmware

Khi ESP32 được cấp nguồn, chương trình bắt đầu khởi tạo các module phần cứng gồm encoder, motor PWM và Ethernet W5500. Sau đó, hệ thống tạo task điều khiển PI để cập nhật tốc độ động cơ liên tục. Đồng thời, chương trình khởi chạy TCP Server để chờ kết nối từ phần mềm máy tính.

Khi máy tính kết nối thành công, ESP32 chờ nhận frame dữ liệu. Sau khi nhận frame, firmware kiểm tra ký tự bắt đầu, ký tự kết thúc và CRC. Nếu frame không hợp lệ, chương trình bỏ qua frame. Nếu frame hợp lệ, chương trình tiến hành giải mã mã lệnh CMD.

Nếu lệnh là ghi PWM, firmware tắt chế độ PI và xuất PWM trực tiếp ra động cơ. Nếu lệnh là đặt tốc độ RPM, firmware lưu giá trị tốc độ đặt. Nếu lệnh là bật PI, firmware bắt đầu điều khiển tốc độ động cơ theo giá trị RPM đặt. Nếu lệnh là đọc RPM, firmware

gửi giá trị tốc độ thực tế về máy tính. Nếu lệnh không được hỗ trợ, firmware trả về trạng thái lỗi.

Sau khi xử lý xong lệnh, ESP32 tạo frame phản hồi và gửi lại cho máy tính. Quá trình này được lặp lại liên tục trong suốt thời gian kết nối TCP còn tồn tại.

Lưu đồ thuật toán firmware

Chương 5

MATLAB SIMULINK – PHÂN TÍCH MÃ NGUỒN CƠ CHẾ GIAO TIẾP VÀ THU THẬP DỮ LIỆU QUA TCP/IP

5.1 Khởi tạo cấu hình và thiết lập kết nối TCP/IP

```
esp_ip = '192.168.1.24';  
esp_port = 5000;  
tcp_obj = tcpclient(esp_ip, esp_port, 'Timeout', 2);
```

Khối lệnh này đóng vai trò thiết lập cấu hình mạng. MATLAB đóng vai trò là TCP Client chủ động gửi yêu cầu kết nối đến TCP Server (được cấu hình trên ESP32) thông qua socket được định danh bởi địa chỉ IP và Port cụ thể. Tham số 'Timeout', 2 giới hạn thời gian chờ kết nối tối đa là 2 giây để tránh việc chương trình bị treo cứng nếu phần cứng không phản hồi.

```
Ts = 0.05;          % Thời gian lấy mẫu (50ms)  
total_time = 3;    % Tổng thời gian lấy dữ liệu (3 giây)  
samples = total_time / Ts;  
time_array = zeros(1, samples);  
rpm_array = zeros(1, samples);  
  
pwm_val = 400;
```

```
data_h = bitshift(pwm_val, -8);
data_l = bitand(pwm_val, 255);
send_esp_cmd(tcp_obj, 2, data_h, data_l);
```

$T_s = 0.05$ quyết định chu kỳ trích mẫu dữ liệu vận tốc từ vi điều khiển (tương đương tần số lấy mẫu $f_s = 20\text{Hz}$).

Hàm `zeros(1, samples)` thực hiện kỹ thuật Cấp phát bộ nhớ trước (Pre-allocation). Trong MATLAB, việc khởi tạo sẵn kích thước mảng tĩnh giúp tối ưu hóa tốc độ thực thi vòng lặp, tránh việc hệ điều hành phải liên tục tái cấp phát bộ nhớ khi mảng tăng dần kích thước theo thời gian thực.

Logic xử lý byte: Vì giá trị PWM cấu hình trên ESP32 là 10-bit (0 - 1023), nó chiếm 2 byte bộ nhớ (16-bit). Giao thức mạng chỉ truyền dữ liệu theo từng byte 8-bit, do đó cần thực hiện toán tử số học bit:

```
bitshift(pwm_val, -8): Dịch 8 bit để lấy 8 bit cao (High Byte).
bitand(pwm_val, 255): Dùng toán tử AND với 255 (0xFF) để lấy 8 bit thấp (Low Byte).
```

Hàm `send_esp_cmd` đóng gói các byte này cùng với mã lệnh `CMD_WRITE_PWM` (0x02) thành một frame 12-byte chuẩn để gửi xuống phần cứng qua socket.

5.2 Vòng lặp lấy mẫu thời gian

for i = 1:samples

```
send_esp_cmd(tcp_obj, 7, 0, 0); % CMD_READ_ENCODER = 0x07
if tcp_obj.NumBytesAvailable >= 12
    rx_data = read(tcp_obj, 12, "uint8");
    if rx_data(1) == 64 && rx_data(12) == 35
        rpm_h = double(rx_data(6));
        rpm_l = double(rx_data(7));
        rpm_val = (rpm_h * 256) + rpm_l;
        if rpm_val > 32767
            rpm_val = rpm_val - 65536;
```

end

```
rpm_array(i) = rpm_val;
```

```
end
```

```
end
```

```
time_array(i) = toc;  
pause(Ts);
```

```
end
```

Mỗi chu kỳ, MATLAB gửi lệnh 0x07 yêu cầu ESP32 phản hồi vận tốc.

Điều kiện NumBytesAvailable ≥ 12 đảm bảo bộ đệm nhận (Rx Buffer) đã nhận đủ độ dài frame quy định trước khi đọc dữ liệu, tránh hiện tượng đọc thiếu byte làm lệch khung dữ liệu.

rx_data(1) == 64 (ýk ựt @) àv rx_data(12) == 35 (ýk ựt #) đóng vai trò là các bit đánh dấu đầu và cuối frame (Header & Footer) để xác thực tính toàn vẹn của gói tin.

rpm_val = (rpm_h * 256) + rpm_l: Dịch ngược byte cao lên 8 bit rồi cộng với byte thấp để tạo lại giá trị vận tốc gốc nguyên 16-bit ban đầu.

Xét âm 2: Điều kiện **if** rpm_val > 32767 dùng để chuyển đổi kiểu dữ liệu **unsigned** 16-bit thu được từ tổng sang kiểu ký số có dấu (Signed Integer). Nếu giá trị vượt ngưỡng của biến unsigned 16-bit, nó sẽ vượt đi 216 = 65536 để chuyển về đúng giá trị âm thực tế (khi động cơ quay ngược).

pause(Ts): Cường bức vòng lặp dừng lại đúng bằng khoảng thời gian 50ms, đảm bảo tính đồng đều của chu kỳ trích mẫu thời gian thực.

Hình 5.1: Hình 5: Đồ thị đáp ứng tốc độ của động cơ

Chương 6

THIẾT KẾ GIAO DIỆN QUẢN LÝ C#

Các thư viện liên quan trong phần kết nối TCP Client

```
using System.Net.Sockets;using System.Threading;using  
    System.Threading.Tasks;using System.IO;
```

Trong đó:

Thư viện

Dùng để làm gì

System.Net.Sockets

Tạo TCP Client, kết nối tới ESP32, lấy luồng truyền nhận dữ liệu

System.Threading

Tạo CancellationTokenSource để hủy vòng lặp nhận dữ liệu

System.Threading.Tasks

Chạy hàm nhận dữ liệu bất đồng bộ ReceiveLoopAsync()

System.IO

Bắt lỗi IOException khi luồng TCP bị đóng hoặc mất kết nối

6.1 Thiết kế giao diện winform

Hình 6.1: Figure 61: Tổng quan giao diện C#

6.2 Cấu hình TCP Client trên C#

Chương trình con khởi tạo thông số kết nối

```
private void Form1_Load(object sender, EventArgs e)
{
    txtIP.Text = "192.168.1.24";
    txtPort.Text = "5000";
}
txtIP.Text = "192.168.1.24"; -- àl địa ỉch IP ủa ESP32 trong ạng LAN.
txtPort.Text = "5000"; -- àl port TCP àm ESP32 đang ỏm để ềch ámy ítnh ểkt ốni.
```

Nhờ đó khi mở phần mềm, người dùng không cần nhập lại IP và port mỗi lần chạy, chỉ cần bấm nút Connect để kết nối.

Chương trình con kết nối TCP Client

```
private void ConnectTcp()
{
    DisconnectTcp(silent: true);

    string host = txtIP.Text.Trim();
    int port = GetPort();

    client = new TcpClient();
    client.NoDelay = true;
    client.Connect(host, port);

    stream = client.GetStream();

    receiveCts = new CancellationTokensource();
    receiveTask = ReceiveLoopAsync(receiveCts.Token);
}
```

```

SetConnectionStatus("Connected");
AppendLog($"Connected to {host}:{port}");
}

```

Hàm ConnectTcp() thực hiện kết nối TCP từ máy tính tới ESP32 thông qua địa chỉ IP và port đã nhập trên giao diện.

6.3 Xây dựng hàm đóng gói frame @SEN

Trước khi xây dựng frame truyền, chương trình định nghĩa độ dài frame và các mã lệnh điều khiển. Mỗi frame truyền/nhận có độ dài cố định là 12 byte. Các mã lệnh CMD dùng để phân biệt chức năng như đọc ADC, ghi PWM, đọc tốc độ encoder, gửi setpoint RPM, gửi hệ số PI và bật/tắt điều khiển PI.

```

private const int FRAME_SIZE = 12; private const byte CMD_READ_ADC =
    0x01; private const byte CMD_WRITE_PWM = 0x02; private const byte CMD_STATUS
    = 0x04; private const byte STATUS_OK = 0x00; private const byte
    CMD_READ_ENCODER = 0x07; private const byte CMD_SET_TARGET_RPM =
    0x08; private const byte CMD_SET_KP = 0x09; private const byte CMD_SET_KI =
    0x0A; private const byte CMD_PI_ENABLE = 0x0B; private const byte
    CMD_READ_PWM = 0x0C;

```

Trong đó:

Hằng số

Ý nghĩa

FRAME_SIZE

Độ dài cố định của frame truyền/nhận, bằng 12 byte

CMD_READ_ADC

Lệnh đọc giá trị ADC

CMD_WRITE_PWM

Lệnh ghi giá trị PWM điều khiển động cơ

CMD_STATUS

Lệnh đọc trạng thái thiết bị

STATUS_OK

Trạng thái phản hồi không lỗi

CMD_READ_ENCODER

Lệnh đọc tốc độ động cơ từ encoder

CMD_SET_TARGET_RPM

Lệnh gửi tốc độ đặt RPM

CMD_SET_KP

Lệnh gửi hệ số Kp

CMD_SET_KI

Lệnh gửi hệ số Ki

CMD_PI_ENABLE

Lệnh bật/tắt bộ điều khiển PI

CMD_READ_PWM

Lệnh đọc giá trị PWM hiện tại

6.4 Chương trình con tính CRC

```
private byte CalcCRC(byte[] frame){ byte crc = 0; for (int i = 0; i <= 9;
    i++) {      crc ^= frame[i]; } return crc;}
```

Hàm CalcCRC() dùng để tính byte kiểm tra lỗi cho frame. CRC được tính bằng phép XOR từ byte 0 đến byte 9. Giá trị CRC sau đó được đưa vào byte thứ 10 của frame để bên nhận kiểm tra dữ liệu có bị sai hay không.

6.5 Chương trình con đóng gói frame @SEN

```
private byte[] BuildSendFrame(byte cmd, ushort data, byte opt1, byte opt2,
    byte seq){ byte[] frame = new byte[FRAME_SIZE]; frame[0] = (byte)'@';
    frame[1] = (byte)'S'; frame[2] = (byte)'E'; frame[3] = (byte)'N'; frame[4]
    = cmd; frame[5] = (byte)(data >> 8); frame[6] = (byte)(data & 0xFF);
    frame[7] = opt1; frame[8] = opt2; frame[9] = seq; frame[10] =
```

```
CalcCRC(frame); frame[11] = (byte) '#'; return frame;}
```

Hàm BuildSendFrame() dùng để tạo frame truyền từ PC xuống ESP32. Frame gửi có dạng:

Frame truyền @SEN:

@ S E N CMD DATA_H DATA_L OPT1 OPT2 SEQ CRC #

Byte	Nội dung	Ý nghĩa
0	@	Ký tự bắt đầu frame
1-3	SEN	Header frame gửi
4	CMD	Mã lệnh cần gửi
5	DATA_H	Byte cao của dữ liệu
6	DATA_L	Byte thấp của dữ liệu
7	OPT1	Byte tùy chọn 1
8	OPT2	Byte tùy chọn 2
9	SEQ	Số thứ tự frame
10	CRC	Mã kiểm tra lỗi
11	#	Ký tự kết thúc frame

6.6 Chương trình con gửi frame xuống ESP32

```
private async Task SendFrameAsync(byte cmd, ushort data){ if (client == null
|| stream == null || !client.Connected) { MessageBox.Show("ưChả ếtkt ốni
TCP"); return; } byte seq = txSeq; txSeq++; byte[] frame =
BuildSendFrame(cmd, data, 0x00, 0x00, seq); await sendLock.WaitAsync();
try { await stream.WriteAsync(frame, 0, frame.Length); await
stream.FlushAsync(); AppendLog($"TX: {ByteArrayToHex(frame)}");
AppendLog($"TX Decode: {DescribeSendFrame(frame)}"); } finally {
sendLock.Release(); }}
```

Hàm SendFrameAsync() có nhiệm vụ gửi frame đã đóng gói xuống ESP32 thông qua NetworkStream. Trước khi gửi, hàm kiểm tra TCP đã kết nối hay chưa. Sau đó chương trình tăng số thứ tự SEQ, tạo frame bằng BuildSendFrame(), rồi ghi dữ liệu xuống luồng

TCP. Biến sendLock được dùng để tránh trường hợp nhiều lệnh gửi đồng thời gây chồng dữ liệu.

6.7 Xây dựng hàm nhận và giải mã frame @REC

Khi ESP32 phản hồi về PC, dữ liệu nhận được có dạng frame @REC. Frame phản hồi cũng có độ dài 12 byte, gồm mã lệnh, dữ liệu, trạng thái xử lý, mã lỗi, số thứ tự và CRC.

Dạng frame nhận:

Frame phản hồi @REC:		
@	R	E
C	CMD	DATA_H
	DATA_L	STATUS
	ERROR	SEQ
	CRC	#

Byte	Nội dung	Ý nghĩa
0	@	Ký tự bắt đầu frame
1-3	REC	Header frame phản hồi
4	CMD	Mã lệnh phản hồi
5	DATA_H	Byte cao của dữ liệu
6	DATA_L	Byte thấp của dữ liệu
7	STATUS	Trạng thái xử lý, 0 là OK
8	ERROR	Mã lỗi nếu có
9	SEQ	Số thứ tự frame
10	CRC	Mã kiểm tra lỗi
11	#	Ký tự kết thúc frame

6.8 Chương trình con nhận dữ liệu từ ESP32

```
private async Task ReceiveLoopAsync(Cancellation_token token){ List<byte>
    rxBuffer = new List<byte>(); byte[] temp = new byte[256]; try {
        if (stream == null) {
            return;
        } while (!token.IsCancellationRequested) {
            int bytesRead = await
            stream.ReadAsync(temp, 0, temp.Length);
            if (bytesRead == 0) {
                break;
            } for (int i = 0; i < bytesRead;
            i++) {
                rxBuffer.Add(temp[i]);
            }
        }
    }
```

```

ProcessRxBuffer(rxBuffer);    }    }    catch (ObjectDisposedException) {
}    catch (IOException) {    }    catch (Exception ex) {
AppendLog($"RX error: {ex.Message}"); }    finally    {        if
(!token.IsCancellationRequested)    {
SetConnectionStatus("Disconnected");    }    }}

```

Hàm `ReceiveLoopAsync()` chạy nền để liên tục nhận dữ liệu từ ESP32. Vì TCP truyền dữ liệu theo dạng luồng byte nên một lần nhận có thể chưa đủ frame hoặc có thể chứa nhiều frame liên tiếp. Vì vậy dữ liệu nhận được sẽ được đưa vào `rxBuffer`, sau đó gọi `ProcessRxBuffer()` để tách frame hợp lệ.

6.9 Chương trình con xử lý bộ đệm nhận

```

private void ProcessRxBuffer(List<byte> rxBuffer){ while (rxBuffer.Count >=
FRAME_SIZE) {    int startIndex = rxBuffer.IndexOf((byte)'@');    if
(startIndex < 0)    {        rxBuffer.Clear();        return;    }
        if (startIndex > 0)    {        rxBuffer.RemoveRange(0,
startIndex);    }    if (rxBuffer.Count < FRAME_SIZE)    {
return;    }    byte[] frame = rxBuffer.GetRange(0,
FRAME_SIZE).ToArray(); bool headerOk =        frame[0] == (byte)'@' &&
        frame[1] == (byte)'R' &&        frame[2] == (byte)'E' &&
frame[3] == (byte)'C' &&        frame[11] == (byte) '#';    if (!headerOk)
    {        rxBuffer.RemoveAt(0);        continue;    }
rxBuffer.RemoveRange(0, FRAME_SIZE);    AppendLog($"RX:
{ByteArrayToHex(frame)}");    if (CheckReceiveFrame(frame))    {
AppendLog($"RX Decode: {DescribeReceiveFrame(frame)}");
CompletePendingResponse(frame);        ParseReceiveFrame(frame);    }
        else    {        AppendLog("RX Error: Frame sai CRC ặhoc sai
định ặdng");    }    }}

```

Hàm `ProcessRxBuffer()` dùng để tìm và tách frame phản hồi @REC trong bộ đệm nhận. Chương trình tìm ký tự bắt đầu @, kiểm tra đủ 12 byte, kiểm tra header @REC và ký tự kết thúc #. Nếu frame hợp lệ về định dạng, chương trình tiếp tục kiểm tra CRC bằng `CheckReceiveFrame()`. Khi frame đúng, dữ liệu sẽ được giải mã bởi `ParseReceiveFrame()`.

6.10 Chương trình con kiểm tra frame nhận

```
private bool CheckReceiveFrame(byte[] frame){ if (frame.Length != FRAME_SIZE)
{   return false; }   if (frame[0] != (byte) '@') return false; if
(frame[1] != (byte) 'R') return false; if (frame[2] != (byte) 'E') return
false; if (frame[3] != (byte) 'C') return false; if (frame[11] !=
(byte) '#') return false; return frame[10] == CalcCRC(frame);}
```

Hàm CheckReceiveFrame() kiểm tra frame nhận có đúng độ dài, đúng header @REC, đúng byte kết thúc # và đúng CRC hay không. Nếu một trong các điều kiện sai, frame sẽ bị loại bỏ để tránh giải mã dữ liệu lỗi.

6.11 Chương trình con giải mã frame nhận

```
private void ParseReceiveFrame(byte[] frame){ byte cmd = frame[4]; ushort data
= (ushort)((frame[5] << 8) | frame[6]); byte status = frame[7]; byte error
= frame[8]; byte seq = frame[9]; if (status != STATUS_OK) {
AppendLog($"Device error. CMD=0x{cmd:X2}, ERR=0x{error:X2}, SEQ={seq}");
return; }   switch (cmd)   {       case CMD_READ_ADC:
AppendLog($"ADC = {data}");           break;       case CMD_WRITE_PWM:
SetPwmValue(data);           AppendLog($"PWM = {data}");
lblPwmValue.Text = $"PWM = {data}";           break;       case CMD_STATUS:
AppendLog($"STATUS = {data}");           break;       case
CMD_READ_ENCODER:           SetRpmValue(data);           AppendLog($"RPM =
{data}");           break;       case CMD_SET_TARGET_RPM:
AppendLog($"Target RPM = {data}");           break;       case CMD_SET_KP:
AppendLog($"Kp = {data / 100.0:0.00}");           break;       case
CMD_SET_KI:           AppendLog($"Ki = {data / 100.0:0.00}");           break;
case CMD_PI_ENABLE:           AppendLog(data == 1 ? "PI ON" : "PI
OFF");           break;       case CMD_READ_PWM:           SetPwmValue(data);
AppendLog($"Read PWM = {data}");           break;       default:
AppendLog($"Unknown CMD = 0x{cmd:X2}");           break;   }}
```

Hàm ParseReceiveFrame() tách các trường dữ liệu trong frame phản hồi. CMD cho biết ESP32 đang phản hồi lệnh nào, DATA là dữ liệu trả về, STATUS cho biết lệnh có xử lý thành công hay không, ERROR là mã lỗi và SEQ là số thứ tự frame. Nếu STATUS

khác STATUS_OK, chương trình ghi log lỗi và không xử lý tiếp. Nếu phản hồi hợp lệ, chương trình cập nhật dữ liệu tương ứng như PWM, RPM, trạng thái PI, hệ số Kp, Ki hoặc trạng thái thiết bị.

```
private string DescribeReceiveFrame(byte[] frame){ byte cmd = frame[4]; ushort
    data = (ushort)((frame[5] << 8) | frame[6]); byte status = frame[7]; byte
    error = frame[8]; byte seq = frame[9]; byte crc = frame[10]; string
    cmdName = GetCommandName(cmd); return $"@REC, CMD={cmdName}, DATA={data},
    STATUS={status}, ERR={error}, SEQ={seq}, CRC=0x{crc:X2}";}
```

Hàm DescribeReceiveFrame() chuyển frame nhận dạng byte sang dạng chuỗi dễ đọc để hiển thị trong khung log. Ví dụ thay vì chỉ thấy dữ liệu HEX, người dùng có thể biết frame đó là lệnh READ_ENCODER, dữ liệu RPM là bao nhiêu, trạng thái có lỗi hay không và CRC bằng bao nhiêu.

6.12 Hiển thị dữ liệu realtime: tốc độ, PWM, trạng thái kết nối

Trong chương trình, dữ liệu realtime gồm tốc độ động cơ, giá trị PWM và trạng thái kết nối được cập nhật trực tiếp lên giao diện WinForms. Các dữ liệu này được lấy từ frame phản hồi @REC do ESP32 gửi về. Sau khi frame được giải mã, chương trình sẽ cập nhật các label, thanh trượt PWM, khung log và biểu đồ tốc độ.

6.13 Chương trình con cập nhật trạng thái kết nối

```
private void SetConnectionStatus(string text){ if (InvokeRequired) {
    BeginInvoke(new Action(() => SetConnectionStatus(text))); return; }
    lblConnection.Text = text;}
```

Hàm SetConnectionStatus() dùng để hiển thị trạng thái kết nối giữa PC và ESP32. Khi kết nối thành công, label sẽ hiển thị Connected; khi mất kết nối hoặc ngắt kết nối, label sẽ hiển thị Disconnected.

Trong hàm có sử dụng InvokeRequired để đảm bảo việc cập nhật giao diện được thực hiện an toàn, kể cả khi hàm được gọi từ luồng nhận dữ liệu TCP.

6.14 Chương trình con ghi log realtime

```
private void AppendLog(string text){ if (InvokeRequired) {  
    BeginInvoke(new Action(() => AppendLog(text))); return; }  
    rtbLog.AppendText($"[{DateTime.Now:HH:mm:ss}]  
    {text}{Environment.NewLine}");}
```

Hàm AppendLog() dùng để ghi thông tin truyền nhận vào khung log rtbLog. Mỗi dòng log có kèm thời gian, giúp người dùng theo dõi quá trình gửi frame @SEN, nhận frame @REC, trạng thái xử lý lệnh, giá trị RPM, PWM và các lỗi nếu có.

Hình 6.2: Figure 62: Kết quả đọc log

6.15 Chương trình con cập nhật tốc độ RPM

```
private void SetRpmValue(int value){ if (InvokeRequired) {  
    BeginInvoke(new Action(() => SetRpmValue(value))); return; }  
    lblRpm.Text = value.ToString(); // ãV ãln chart  
    AddChartPoint(currentTargetRpm, value);}
```

Hàm SetRpmValue() dùng để cập nhật tốc độ thực tế của động cơ lên label lblRpm. Giá trị này được lấy từ phản hồi của ESP32 khi PC gửi lệnh đọc encoder.

Ngoài ra, hàm còn gọi AddChartPoint(currentTargetRpm, value) để đưa đồng thời giá trị đặt và giá trị RPM thực tế lên biểu đồ realtime. Nhờ đó người dùng có thể quan sát khả năng bám tốc độ của động cơ theo thời gian.

6.16 Chương trình con cập nhật giá trị PWM

```
private void SetPwmValue(int value){ value = Math.Max(trbPWM.Minimum,  
    Math.Min(trbPWM.Maximum, value)); if (InvokeRequired) { BeginInvoke(new  
    Action(() => SetPwmValue(value))); return; } suppressPwmScroll =  
    true; trbPWM.Value = value; suppressPwmScroll = false;}
```

Hàm SetPwmValue() dùng để cập nhật giá trị PWM lên thanh trượt trbPWM. Giá

trị PWM được giới hạn trong khoảng nhỏ nhất và lớn nhất của thanh trượt, ví dụ từ 0 đến 255.

Biến `suppressPwmScroll` dùng để tránh trường hợp chương trình tự cập nhật thanh trượt nhưng lại kích hoạt sự kiện `trbPWM_Scroll()` và gửi lệnh PWM ngoài ý muốn.

Hình 6.3: Figure 63: Cập nhật PWM manual

6.17 Chương trình con đọc tốc độ RPM

```
private async Task ReadRpmAsync(){ await SendFrameAsync(CMD_READ_ENCODER, 0);}
```

Hàm `ReadRpmAsync()` gửi lệnh `CMD_READ_ENCODER` xuống ESP32 để yêu cầu đọc tốc độ động cơ. Sau khi ESP32 phản hồi frame `@REC`, chương trình sẽ giải mã dữ liệu trong `ParseReceiveFrame()` và cập nhật giá trị RPM lên giao diện.

6.18 Chương trình con tự động đọc RPM

```
private void btnAutoRpm_Click(object sender, EventArgs e){ autoReadRpm =  
    !autoReadRpm; if (autoReadRpm) {      autoRpmTimer.Start();  
    btnAutoRpm.Text = "ỪDng ựt độnđ đọc";    AppendLog("ấBt đầu ựt độnđ đợc  
    RPM"); } else {      autoRpmTimer.Stop();    btnAutoRpm.Text = "ựT  
    độnđ đợc RPM";    AppendLog("ừDng ựt độnđ đợc RPM"); }}
```

Hàm `btnAutoRpm_Click()` dùng để bật hoặc tắt chế độ tự động đọc tốc độ. Khi bật, timer `autoRpmTimer` sẽ chạy định kỳ và gọi hàm `ReadRpmAsync()` để liên tục cập nhật RPM. Chức năng này giúp giao diện hiển thị tốc độ động cơ theo thời gian thực.

Hình 6.4: Figure 64: Dữ liệu rpm realtime

6.19 Đoạn xử lý dữ liệu realtime trong frame phản hồi @REC


```

private void ParseReceiveFrame(byte[] frame){ byte cmd = frame[4]; ushort data
    = (ushort)((frame[5] << 8) | frame[6]); byte status = frame[7]; byte error
    = frame[8]; byte seq = frame[9]; if (status != STATUS_OK) {
    AppendLog($"Device error. CMD=0x{cmd:X2}, ERR=0x{error:X2}, SEQ={seq}");
    return; } switch (cmd) { case CMD_WRITE_PWM:
    SetPwmValue(data); AppendLog($"PWM = {data}");
    lblPwmValue.Text = $"PWM = {data}"; break; case
    CMD_READ_ENCODER: SetRpmValue(data); AppendLog($"RPM =
    {data}"); break; case CMD_READ_PWM: SetPwmValue(data);
    AppendLog($"Read PWM = {data}"); break; case
    CMD_STATUS: AppendLog($"STATUS = {data}"); break;
    default: AppendLog($"Unknown CMD = 0x{cmd:X2}"); break; }}

```

Đoạn chương trình trên là phần giải mã dữ liệu realtime từ ESP32. Khi nhận được phản hồi CMD_READ_ENCODER, chương trình cập nhật tốc độ RPM. Khi nhận được phản hồi CMD_WRITE_PWM hoặc CMD_READ_PWM, chương trình cập nhật giá trị PWM. Các thông tin này đồng thời được ghi vào khung log để phục vụ giám sát và kiểm tra truyền thông.

Chương 7

TỔNG KẾT

Dề tài đã thực hiện nghiên cứu và xây dựng hệ thống card giao tiếp Ethernet sử dụng ESP32 kết hợp module W5500 nhằm phục vụ truyền nhận dữ liệu, giám sát và điều khiển thiết bị từ máy tính. Hệ thống được thiết kế theo mô hình máy tính đóng vai trò thiết bị điều khiển, giám sát; ESP32 đóng vai trò thiết bị nhúng thực hiện xử lý lệnh, đọc tín hiệu phản hồi và điều khiển ngoại vi.

Trong hệ thống, module W5500 được sử dụng để bổ sung khả năng giao tiếp Ethernet cho ESP32 thông qua chuẩn SPI. Nhờ đó, ESP32 có thể kết nối vào mạng LAN bằng cổng RJ45 và giao tiếp với phần mềm trên máy tính thông qua địa chỉ IP và port. Trên nền Ethernet, hệ thống sử dụng giao thức TCP/IP để thiết lập kết nối ổn định giữa phần mềm C# và ESP32.

Một nội dung quan trọng của đề tài là việc xây dựng giao thức truyền thông ứng dụng riêng dạng frame cố định 12 byte. Frame truyền từ máy tính xuống ESP32 được định nghĩa với header @SEN, còn frame phản hồi từ ESP32 về máy tính được định nghĩa với header @REC. Cấu trúc frame bao gồm mã lệnh, dữ liệu, trạng thái, số thứ tự frame và mã kiểm tra lỗi CRC/XOR. Việc sử dụng frame cố định giúp quá trình truyền nhận dữ liệu có cấu trúc rõ ràng, dễ kiểm tra lỗi và phù hợp với các hệ thống điều khiển nhúng.

Phần mềm C# WinForms được xây dựng để đóng vai trò TCP Client, cho phép người dùng nhập địa chỉ IP, port, thực hiện kết nối đến ESP32, gửi lệnh điều khiển và nhận dữ liệu phản hồi. Giao diện C# có các chức năng cơ bản như kết nối/ngắt kết nối TCP, gửi lệnh, điều khiển LED, điều chỉnh PWM bằng thanh trượt, hiển thị trạng thái và log dữ liệu truyền nhận. Trong quá trình phát triển, phần mềm Hercules được sử dụng để giả lập TCP Server, giúp kiểm thử giao diện C# và kiểm tra frame truyền thông trước khi kết nối với phần cứng ESP32 thật.

Về mặt ứng dụng, hệ thống được định hướng điều khiển động cơ DC có encoder. ESP32 có nhiệm vụ xuất tín hiệu PWM điều khiển driver động cơ, đọc xung encoder để xác định tốc độ thực tế và gửi dữ liệu phản hồi về máy tính. Từ đó, hệ thống có thể triển khai bộ điều khiển PID để điều khiển tốc độ động cơ. Bộ điều khiển PID có thể được thực hiện trên ESP32 để đảm bảo tính thời gian thực, hoặc triển khai trên C#/Matlab Simulink để thuận tiện cho việc mô phỏng, giám sát và điều chỉnh tham số.

Nhìn chung, đề tài đã kết hợp nhiều nội dung quan trọng gồm truyền thông Ethernet, mô hình TCP/IP, lập trình ESP32, giao tiếp SPI với W5500, lập trình giao diện C#, thiết kế frame truyền thông và điều khiển động cơ DC có phản hồi encoder. Hệ thống có ý nghĩa thực tiễn trong việc xây dựng một card giao tiếp Ethernet phục vụ các bài toán đo lường, giám sát và điều khiển thiết bị trong mạng LAN.

Chương 8

TÀI LIỆU THAM KHẢO

- [1] Espressif Systems, ESP32 Series Datasheet. Espressif Systems.Link: https://www.espressif.com/sites/default/files/ESP32-Series-Datasheet_en
- [2] WIZnet, W5500 Datasheet. WIZnet Co., Ltd.Link: <https://docs.wiznet.io/Product/iEthernet/W5500/>
- [3] Microsoft, TcpClient Class – System.Net.Sockets. Microsoft Learn.Link: <https://learn.microsoft.com/en-us/dotnet/api/system.net.sockets.tcpclient>
- [4] J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 8th ed. Pearson, 2021.
- [5] MathWorks, UDP Communication in Simulink / UDP Send and UDP Receive Blocks. MathWorks Documentation.Link: <https://www.mathworks.com/help/udpcomm/udp-communication-in-simulink.html>